

```

% =====
% Waveguide Propagation Loss Extraction using Cut-Back Method
% CLEAN CORRECTED VERSION FOR TE MODE
% Improved Figure Size + Smaller Legends
% =====

clear;
clc;
close all;

% Stop execution exactly at error line if any future error occurs
dbstop if error

% =====
% CHECK IF REGRESSION TOOLBOX EXISTS
% =====
Error_Intervals = exist('regress','file');

FONTSIZE = 13;
LEGENDSIZE = 8;

% =====
% DEVICE INFORMATION
% =====
deviceName = 'Spiral waveguide (TE)';

% Wavelength of interest
lambda0 = 1.55e-6;

% =====
% FILE NAMES
% =====
files = { ...
    'LukasC_SpiralWG0kTE_1297.mat', ...
    'LukasC_SpiralWG5kTE_1300.mat', ...
    'LukasC_SpiralWG10kTE_1298.mat', ...
    'LukasC_SpiralWG30kTE_1299.mat' ...
};

% Number of DUTs / waveguide lengths
Num = [0, 0.5, 1.0, 3.0];

% Detector port number
PORT = 2;

% =====
% DROPBOX URLS
% =====
url = { ...

```

```

    'https://www.dropbox.com/s/1v5wzon5nexggn6/LukasC_SpiralWG0kTE_1297.mat?
dl=1', ...
    'https://www.dropbox.com/s/7r8svd1ukjae8ox/LukasC_SpiralWG5kTE_1300.mat?
dl=1', ...
    'https://www.dropbox.com/s/kfsv1yeghue0i38/LukasC_SpiralWG10kTE_1298.mat?
dl=1', ...
    'https://www.dropbox.com/s/5o66l3xd7rnxtgq/LukasC_SpiralWG30kTE_1299.mat?
dl=1' ...
    };

% =====
% DOWNLOAD FILES IF NOT PRESENT
% =====
if ~exist(files{1}, 'file')

    disp('Loading files from Dropbox')

    for i = 1:length(files)

        fprintf('Downloading file %d of %d...\n', i, length(files));

        websave(files{i}, url{i});

    end

else

    disp('Loading files from local disk')

end

```

Loading files from local disk

```

% =====
% INITIALIZE ARRAYS
% =====
amplitude = [];
amplitude_poly = [];
LegendText = {};

% =====
% PLOT RAW DATA + POLYFIT
% =====
figure('Position',[100 100 1200 700]);

for i = 1:length(files)

    % Load MAT file
    load(files{i});

```

```

% Force wavelength into column vector
lambda = scandata.wavelength(:);

% Extract detector data
temp_power = scandata.power(:,PORT);

% Force column vector
temp_power = temp_power(:);

% Store detector data
amplitude(:,i) = temp_power;

% Plot raw data
plot(lambda*1e6, amplitude(:,i));
hold on;

% Polynomial fitting
xfit = (lambda - mean(lambda))*1e6;

p = polyfit(xfit, amplitude(:,i), 4);

amplitude_poly(:,i) = polyval(p, xfit);

% Plot fitted data
plot(lambda*1e6, amplitude_poly(:,i), 'LineWidth',2);

% Legend entries
LegendText{2*i-1} = ...
    ['raw data: ' strrep(files{i}, '_', '\_')];

LegendText{2*i} = ...
    ['fit data: ' strrep(files{i}, '_', '\_')];

end

title(['Optical spectra for the ' deviceName ' test structures']);

xlabel('Wavelength (\mum)');

ylabel('Optical Power (dBm)');

lgd = legend(LegendText, 'Location', 'eastoutside');

set(lgd, 'FontSize', LEGENDSIZE)

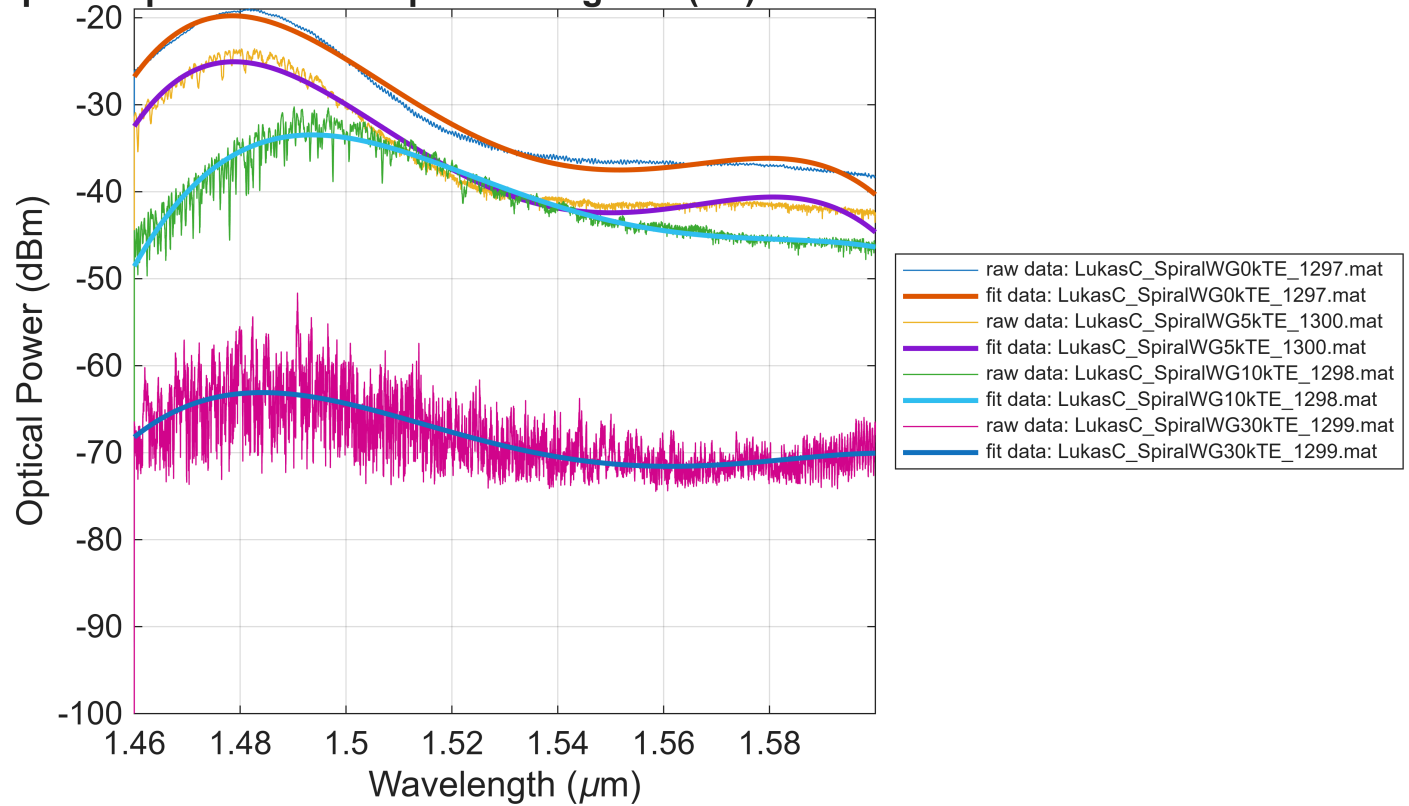
axis tight;

grid on;

set(gca, 'FontSize', FONTSIZE)

```

Optical spectra for the Spiral waveguide (TE) test structures



```
% =====
% LINEAR REGRESSION AT lambda0
% =====

% Find nearest wavelength index
[~, index] = min(abs(lambda - lambda0));

% Linear regression
A = [ones(length(Num),1) Num'] \ amplitude_poly(index,:);

% =====
% PLOT CUT-BACK FIT
% =====
figure('Position',[120 120 1000 650]);

plot(Num, amplitude(index,:), 'x', 'MarkerSize',10);
hold on;

plot(Num, amplitude_poly(index,:), 'o', 'MarkerSize',8);

plot(Num, A(1) + Num*A(2), 'LineWidth',3);

legend( ...
    'raw data at lambda0', ...
```

```

    'polyfit of raw data', ...
    'linear regression of polyfit', ...
    'Location','best', ...
    'FontSize',LEGENDSIZE);

xlabel('Waveguide length (cm)');

ylabel('Loss (dB/cm)');

title(['Cut-back method, ' deviceName ...
    ' Loss, at ' num2str(lambda0*1e9) ' nm']);

grid on;

set(gca,'FontSize',FONTSIZE)

% =====
% CONFIDENCE INTERVALS
% =====
if Error_Intervals

    [b, bint] = regress( ...
        amplitude_poly(index,:)', ...
        [Num' ones(numel(Num),1)]);

    SlopeError95CI = diff(bint(1,:))/2;

    InterceptError95CI = diff(bint(2,:))/2;

    a = annotation('textbox', [.18 .18 .1 .1], ...
        'String', ...
        { ...
        ['Fit results, with 95% confidence intervals:'], ...
        ['slope = ' num2str(A(2),'%0.2g') ...
        ' +/- ' num2str(SlopeError95CI,'%0.01g') ' dB/cm'], ...
        ['intercept = ' num2str(A(1),'%0.3g') ...
        ' +/- ' num2str(InterceptError95CI,'%0.02g') ' dB'] ...
        });

    set(a,'FontSize',11);

    disp(['Cut-back method, ' deviceName ...
        ' Loss at ' num2str(lambda0*1e9) ...
        ' nm = ' num2str(-A(2),'%0.2g') ...
        ' +/- ' num2str(SlopeError95CI,'%0.01g') ...
        ' dB/cm'])

else

    disp('Skipping fitting error estimations')

```

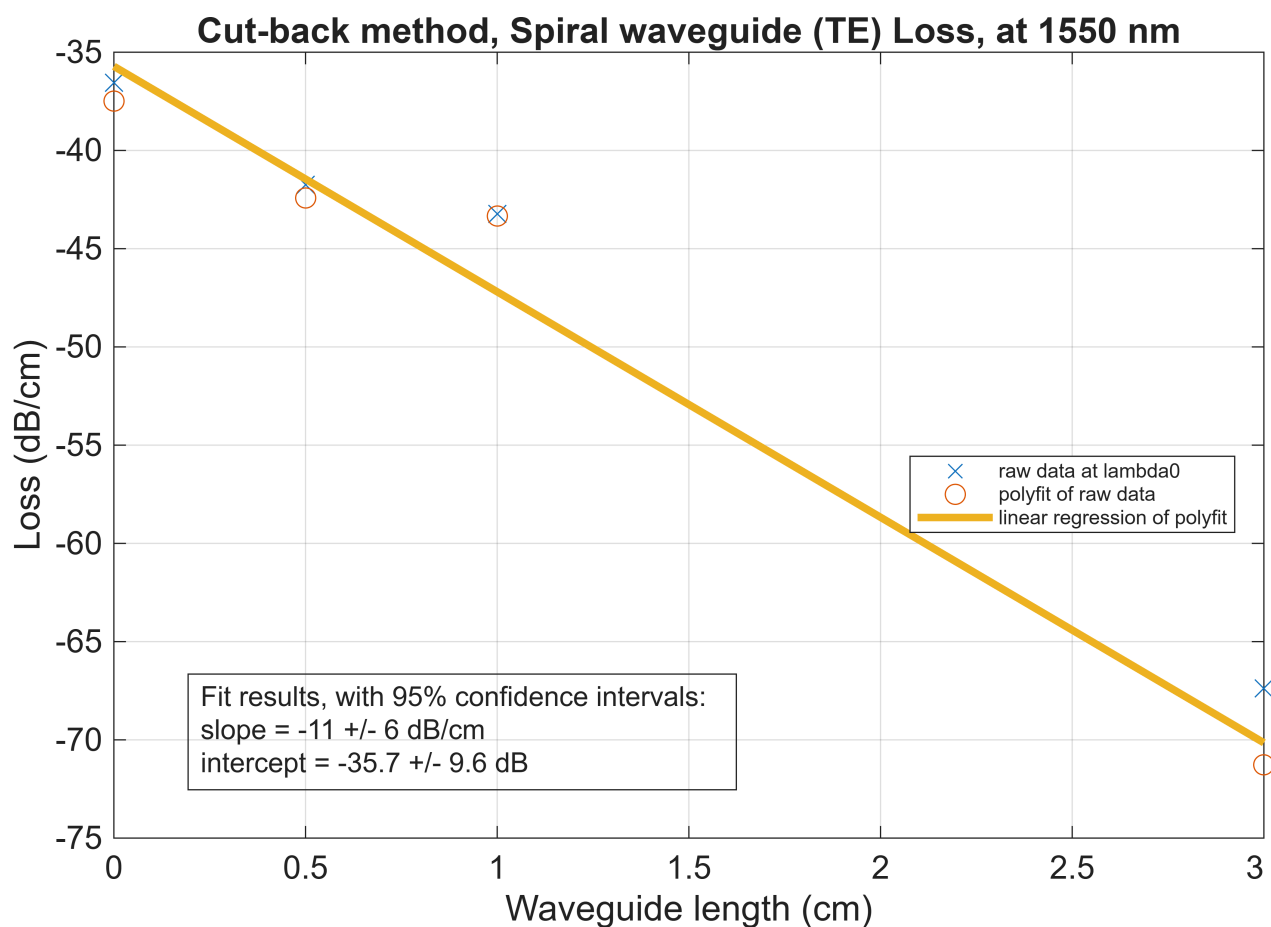
```

annotation('textbox', [.18 .18 .1 .1], ...
    'String', ...
    { ...
    ['Fit results:'], ...
    ['slope = ' num2str(A(2), '%0.2g') ' dB/cm'], ...
    ['intercept = ' num2str(A(1), '%0.3g') ' dB'] ...
    });

disp(['Cut-back method, ' deviceName ...
    ' Loss at ' num2str(lambda0*1e9) ...
    ' nm = ' num2str(-A(2), '%0.2g') ...
    ' dB/cm'])

```

end



Cut-back method, Spiral waveguide (TE) Loss at 1550 nm = 11 +/- 6 dB/cm

```

% =====
% WAVELENGTH DEPENDENCE OF LOSS
% =====

% Linear regression using raw data
C = [ones(length(Num),1) Num'] \ amplitude';

```

```

figure('Position',[140 140 1200 700]);

if Error_Intervals

    slope = [];
    slope_int = [];

    lambda_downsampled = lambda(1:100:end);

    amplitude_poly_downsampled = ...
        amplitude_poly(1:100:end,:);

    for i = 1:length(lambda_downsampled)

        [b, bint] = regress( ...
            amplitude_poly_downsampled(i,:)', ...
            [Num' ones(numel(Num),1)]);

        slope(i) = b(1);

        slope_int(i,:) = bint(1,:);

    end

    % Confidence interval shaded region
    X = [lambda_downsampled; flipud(lambda_downsampled)]*1e6;

    Y = [slope_int(:,1); flipud(slope_int(:,2))];

    fill(X, -Y, [0.7 1 1], ...
        'LineStyle','none');

    hold on;

    % Polyfit-based result
    plot(lambda_downsampled*1e6, ...
        -slope', ...
        'b', ...
        'LineWidth',3);

    % Raw-data result
    plot(lambda*1e6, ...
        -C(2,:) ', ...
        'LineWidth',1.5);

    legend( ...
        'Loss, 95% Confidence Interval', ...
        'Loss from polyfit', ...
        'Loss from raw data', ...

```

```

        'Location','best', ...
        'FontSize',LEGENDSIZE);

else

    D = [ones(length(Num),1) Num'] \ amplitude_poly';

    plot(lambda*1e6, ...
        -[D(2,:) C(2,:)]);

    legend( ...
        'Loss from polyfit', ...
        'Loss from raw data', ...
        'Location','best', ...
        'FontSize',LEGENDSIZE);

end

axis tight;

yl = ylim;

ylim([0 yl(2)])

title(['Cut-back method, ' deviceName ...
    ' Loss wavelength dependence'])

ylabel('Loss (dB/cm)');

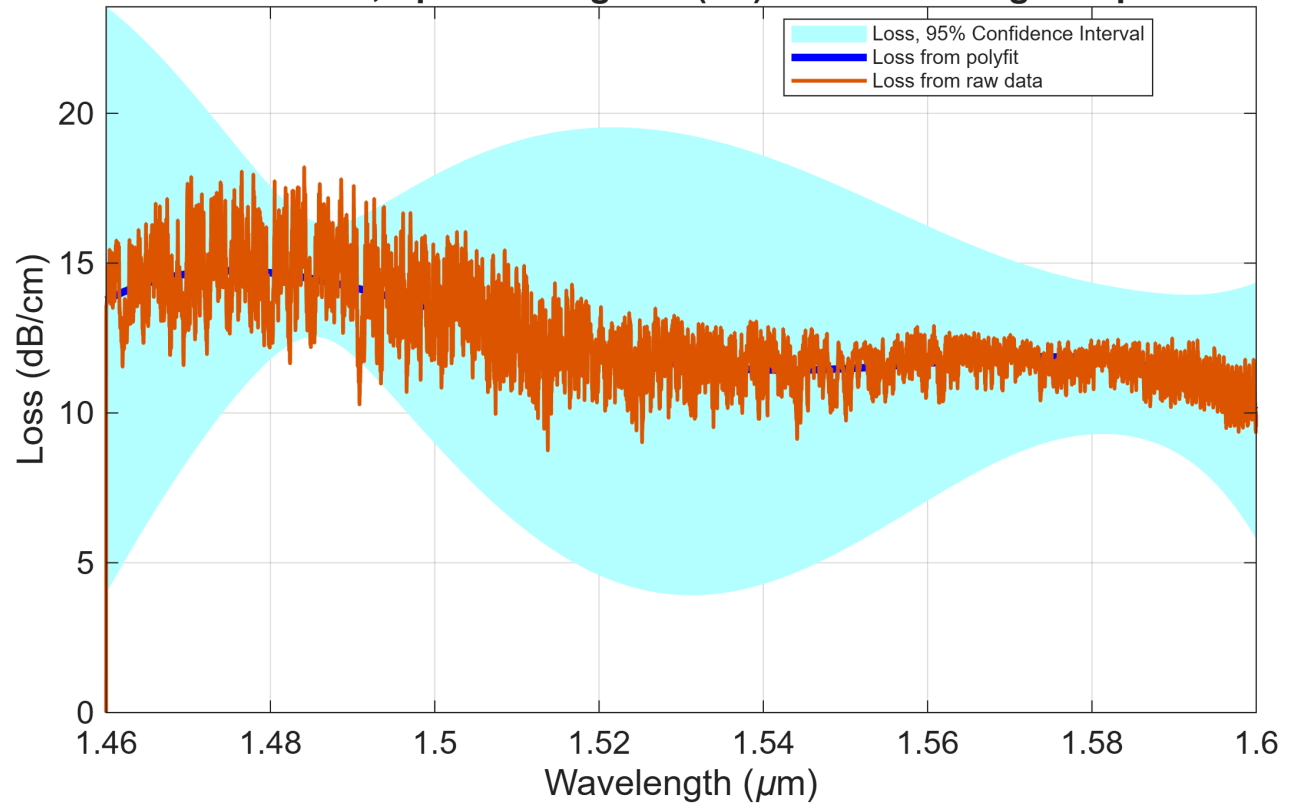
xlabel('Wavelength (\mum)');

grid on;

set(gca, 'FontSize', FONTSIZE)

```


Cut-back method, Spiral waveguide (TE) Loss wavelength dependence



```
disp('Program completed successfully.')
```

Program completed successfully.